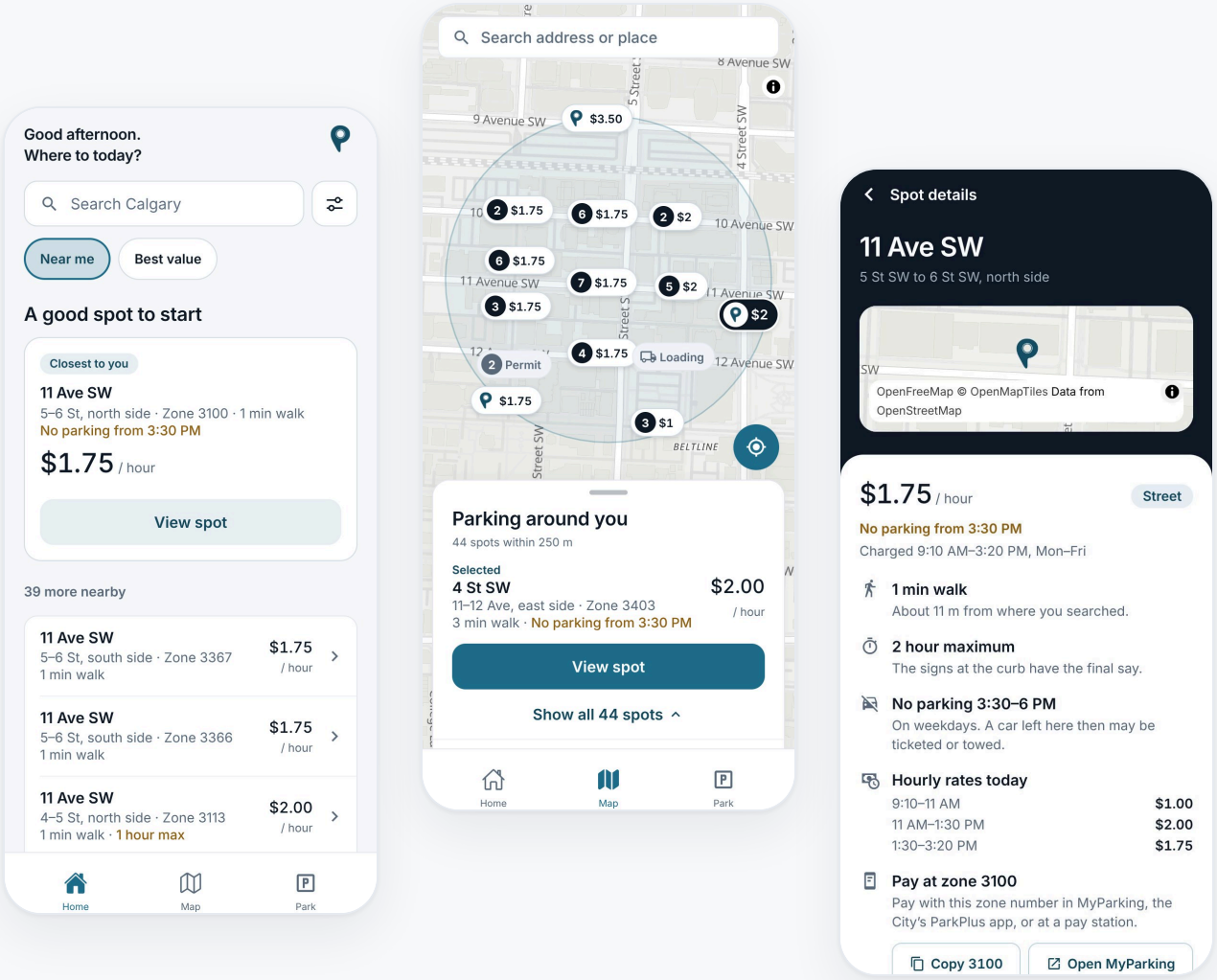
CASE STUDY · MOBILE APP

ParkAid

Street parking in downtown Calgary, with the City's real rates, time limits and rush-hour rules.

Role	Capstone team member (team of five), then the follow-up work on my own, with AI coding help
Build	React Native (Expo SDK 54) for iOS, Android and web. Node and Express API, Postgres with PostGIS.
Timeline	SAIT capstone, then September 2025 to November 2026

Source code	github.com/ginesbal/parkaid
Demo	Seven recordings of the real build
Figma	Design file: components and screens



Ehrl Balquin · linkedin.com/in/ehrlbalquin

02 · AT A GLANCE

Real rates and rules, in the order a driver needs them

The problem

Calgary publishes each block's zone, rates by time of day and rush-hour rules as open data. The capstone version guessed prices from zone codes, showed raw City addresses and listed the same block several times.

What I did

Rebuilt the data path and the interface around four questions: can I park here now, what does it cost right now, until when, and which zone do I pay? Then three rounds of critique, each followed by small, tested fixes.

The result

Rates and rush-hour rules come straight from the City's data. A busy downtown map reads at a glance. Payment is plain: ParkAid keeps time, and MyParking, the City's app, takes the money.

23.5%

of downtown street rows were repeats of the same stretch (277 of 1,178). Each now comes back once.

45 → 15

pins on the opening map view. 44 overlapped before; 4 touch now.

170

automated tests passing: 152 in the app, 18 in the backend.

4.23 → 4.74

contrast ratio for grey text on a selected row, now above the 4.5 : 1 AA minimum.

03 The problem
04 My contribution
05 Process
06 Before and after
07 Key decisions
08 Map and filters
09 React Native build
10 Technical challenges
11 Testing and results
12 Walkthrough
13 Figma file
14 Outcome, gaps, links

The City's data was right. The capstone app read it loosely.

Downtown parking changes by block, by side of the street and by the hour. Before leaving the car, a driver needs four answers:

1. Can I park here now?
2. What does it cost right now?
3. Until when? Time limits, rush-hour tow-away.
4. Which zone do I pay in MyParking?

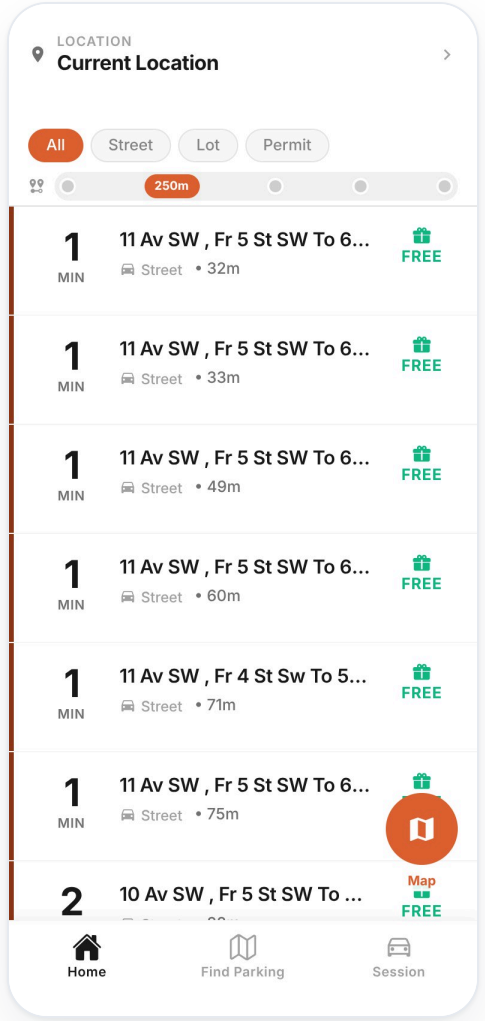
The City publishes all of this as open data. The capstone code, as of October 2025:

Guessed prices. The price came from a lookup on the zone's code, not the City's rates. Zone 1684 showed $6.00 an hour; the City charges $2.00 to $2.25.

Wrong "FREE". Blocks on 11 Ave SW, which the City charges for on weekdays, showed FREE.

Hard to read. Rows used raw City addresses ("11 Av SW , Fr 5 St SW To 6 St SW"), and the same block appeared several times.

Not real yet. The web build showed a static map image, and the session screen was a demo with rates you picked yourself.



Capstone code, October 2025, run with the City's current records. Its pricing logic was re-created from the old backend.

From a team capstone to a product I kept improving

CAPSTONE · TEAM OF FIVE

SAIT, Year 2, Semester 2

We built the base app together: an Expo app with a home list, a map and a parking session, a Node and Express API, and an import of the City's parking data.

This repo starts from that finished code (first commit September 29, 2025), so the team's own commit history isn't in it.

AFTER THE CAPSTONE · SEPTEMBER 2025 TO NOVEMBER 2026

My own work, with AI coding help

- Made the backend deployable: Render with push-to-deploy, Supabase Postgres with PostGIS, one required setting.
- Replaced guessed prices with the City's rates by time of day, and added rush-hour restrictions.
- Merged repeated segments and placed each block's pin at its midpoint.
- Redesigned the app: a calmer palette and type scale, block names that match the signs, and a plain payment flow.
- Put a real map on the web and grouped pins that overlap.
- Ran three rounds of design critique and shipped the fixes in small, tested steps.

101 of 128

commits were written by Claude Code, an AI coding agent, from March 2026 on, working from my instructions.

27

commits are under my own accounts. A few came from an OpenAI Codex branch merged in April 2026.

Every round

I set the priorities and made the product calls: what to fix, in what order, and what the app should promise.

Look at the running app, decide, fix, check

Year 2 · capstone

Base app

Team of five builds the app, API and data import.

September 2025

New repo

The finished capstone code moves to this repo.

March to May 2026

Redesign and deploy

New interface; backend live on Render; renamed ParkAid.

October 2026

Critique and fix

Three critique rounds. The last one became ten fix steps.

November 2026

Show the work

Real-data recordings, the Figma file and this case study.

How each round worked

1 · Capture

Run the real build

The web build with the City's records for the blocks around 11 Ave SW and 5 St SW, at a fixed time (a Tuesday, 1:40 PM). Screenshot every screen.

2 · Critique

Review it

Usability heuristics, wording, and accessibility: contrast, keyboard order, screen-reader labels, small and wide screens.

3 · Decide

Choose the order

My call for the last round: money facts first (rates, rush hour, cost), load the City's real rates, and fix the backend too.

4 · Fix and check

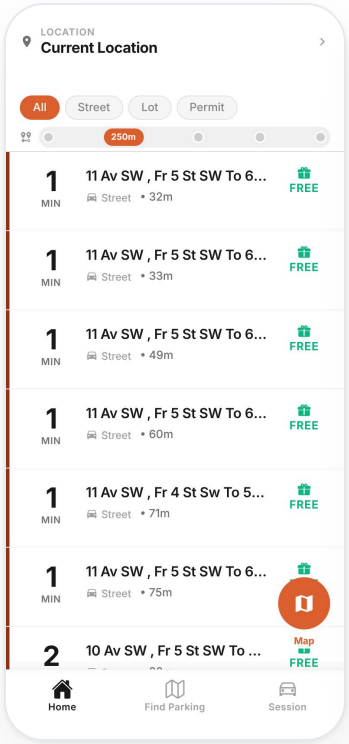
One step at a time

Each fix in its own commit, with Jest tests, the web build checked at 360, 390 and 1280 px, and an Android bundle build.

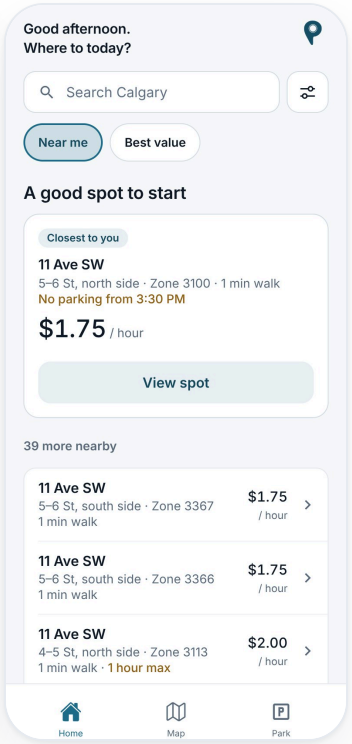
The critiques were AI-assisted reviews of the running build, not sessions with drivers.

Same City data, clearer answers

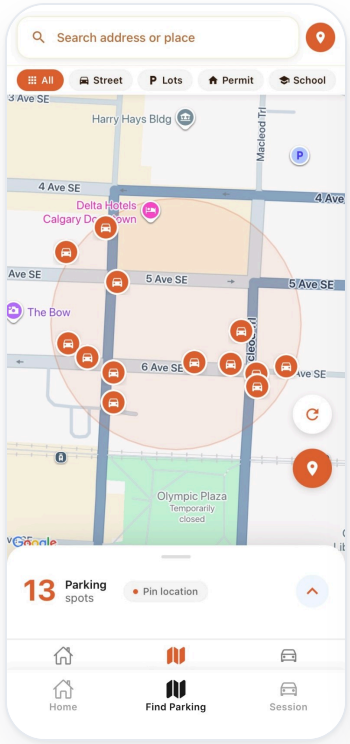
BEFORE



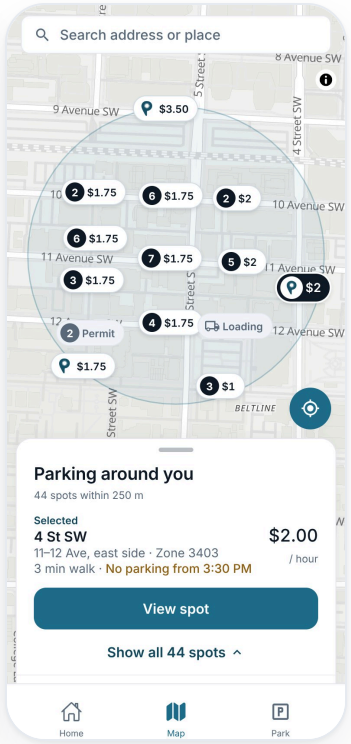
AFTER



BEFORE



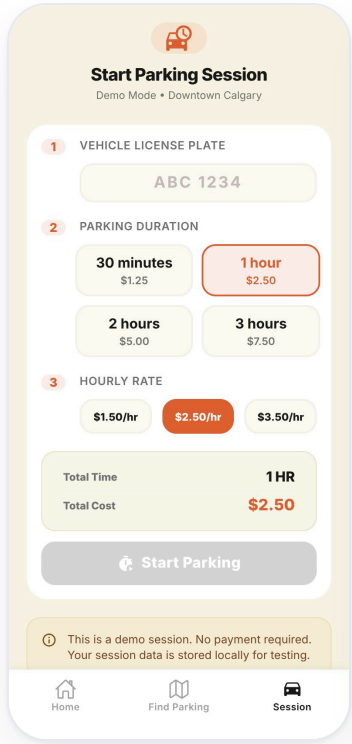
AFTER



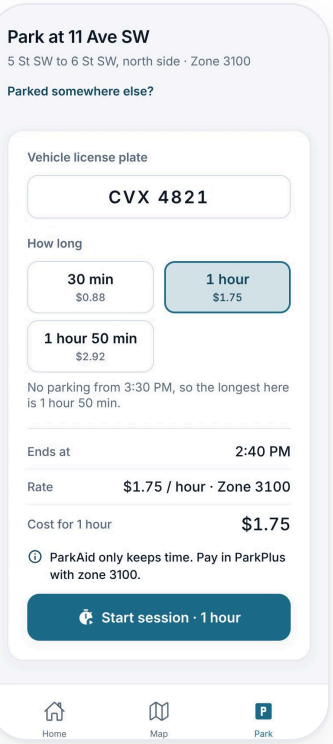
Home. Raw addresses and FREE on charged blocks became block, side and zone in plain words, the rate now, and the rush-hour warning in amber.

Map. A static image on the web build with pins that show no price became a real map, with overlapping pins grouped by count and lowest rate.

BEFORE



AFTER



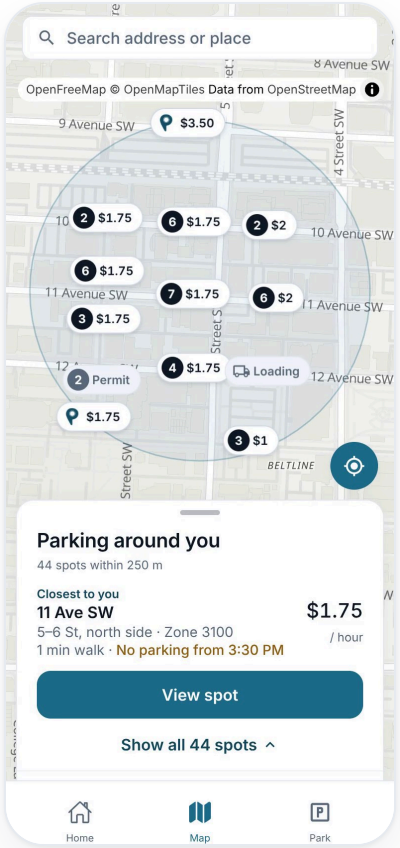
Park. A demo where you chose the hourly rate became the City's rate, the end time and the cost before you start, and a pointer to pay in ParkPlus.

Before: the capstone code from October 2025, run locally. After: the current web build. Both use the City's records for the same downtown blocks.

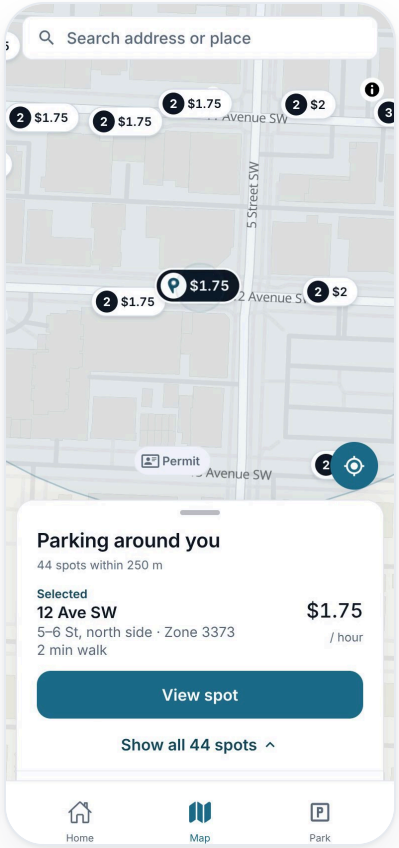
Five calls that shaped the app

1	**Show the City's rate for right now, and the rest of the day**	One price is wrong for most of the day. On 11 Ave SW the rate is $1.00 until 11 AM, $2.00 until 1:30 PM, then $1.75.	**WHERE** Rows, map pins, Details
2	**Treat rush hour as a rule, not a note**	A car left in a tow-away window gets ticketed or towed. So the list warns, lengths stop before the window, and Start isn't offered during it.	**WHERE** Home, Details, Park
3	**Be plain about money**	ParkAid doesn't take payment. It times the stay, shows the cost, copies the zone, opens MyParking, and reminds you to end that session too.	**WHERE** Park, session, summary
4	**Name blocks the way the signs do**	"5–6 St, north side · Zone 3100" matches what a driver sees at the curb. Two zones on one block are told apart by side.	**WHERE** Rows and cards
5	**Group pins that would overlap**	Downtown, 45 blocks fit within 250 m. One pin shows the count and the lowest rate; a tap zooms in until the spots part.	**WHERE** Map, on phones and web

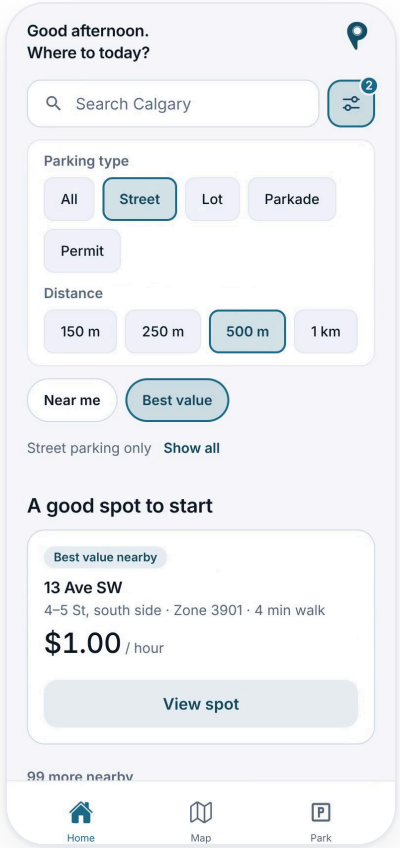
Compare spots without tapping every pin



Opening view. A group shows its count and its lowest rate.



After a tap on a group. The map zooms until the spots part; the chosen pin turns dark.



Filters. Best value, street only, within 500 m.

Sort

Near me ranks by walk, then price. **Best value** ranks by the rate now, then distance. In both, short stays, blocks with no parking right now, and permit-only blocks go lower.

Filter

Parking type (street, lot, parkade, permit) and distance (150 m to 1 km). The list and the map share the same filters.

Move around

Search an address, drop a pin, or drag the map and search that area.

Grouping rule

Pins closer than a group pill (70 × 34 px at the current zoom) share one pin, led by the best-ranked spot. If every spot in a group is within 12 m, a tap selects the lead instead of zooming.

One codebase, three platforms, two map engines

City of Calgary open data
Parking zones, rates by time of day, restrictions

↓

Import script (Node)
Four datasets into one table; safe to re-run

↓

Postgres + PostGIS on Supabase
Nearby search runs in the database, with an index

↓

Express API on Render
Merges a block's segments, formats rows

↓

iOS and Android
react-native-maps (Google)

Web
react-native-web, MapLibre GL JS, OpenStreetMap tiles

My front-end responsibilities

- **Screens and navigation:** Welcome, Home, Map, Details, Park and the session, with React Navigation.
- **Shared components:** spot rows, the meta line that wraps by part, map pins and group pins, ParkPlus actions.
- **Rules kept out of the UI:** rates, rush hour, ranking and grouping live in plain functions with Jest tests.
- **Platform files:** index.web.js for the web map; a compact badge for Android's marker limit.
- **State:** a session context for the timer, hooks for location and nearby spots, and cached responses for bad connections.
- **Access and layout:** screen-reader labels, Reduce Motion, keyboard order on web, a 520 px column on wide screens.

Expo SDK 54 · React Native 0.81 · React Navigation · react-native-maps · MapLibre GL JS · Jest

```
// mobile/src/utils/mapGroups.js (simplified)
export function groupPins(spots, worldSize) {
  const groups = [];
  for (const spot of spots) { // best first
    const { x, y } = worldPoint(
      getCoordinates(spot), worldSize);
    const near = groups.find((g) =>
      Math.abs(g.x - x) < PIN_WIDTH &&
      Math.abs(g.y - y) < PIN_HEIGHT);
    if (near) near.spots.push(spot);
    else groups.push(
      { x, y, lead: spot, spots: [spot] });
  }
  return groups;
}
```

The same grouping runs on both map engines. Pin positions come from Web Mercator, so only the screen distance between pins matters.

What was hard, and how it was solved

Prices that change by the hour

APPROACH

Read the City's zone-rate layer, turn its rate bands into time ranges, and count only the hours the City charges.

RESULT

Rows show the rate now, Details lists the day's rates, and Park shows the cost before you start.

Rush-hour restrictions

APPROACH

Check each restriction's days and hours against the current time and against the end of a planned stay.

RESULT

Amber warnings, capped lengths ("the longest here is 1 hour 50 min"), and no Start during the window.

Repeated segments

APPROACH

The City splits a stretch of a block into segments. The API merges rows with the same zone, address, side, type, hours, limit and restriction.

RESULT

277 of 1,178 street rows near downtown were repeats. Each block now comes back once, pinned at its midpoint.

A crowded map

APPROACH

Group pins by their distance on screen at the current zoom, on both map engines, and zoom in on tap.

RESULT

45 pins with 44 overlapping became 15 pins with 4 touching on the opening web view.

Android map markers

APPROACH

react-native-maps on Android draws a custom marker into a 100 × 100 px bitmap, which crops a price pill. Android gets a compact round badge instead.

RESULT

No cropped pins. The selected spot's price shows in the sheet below the map.

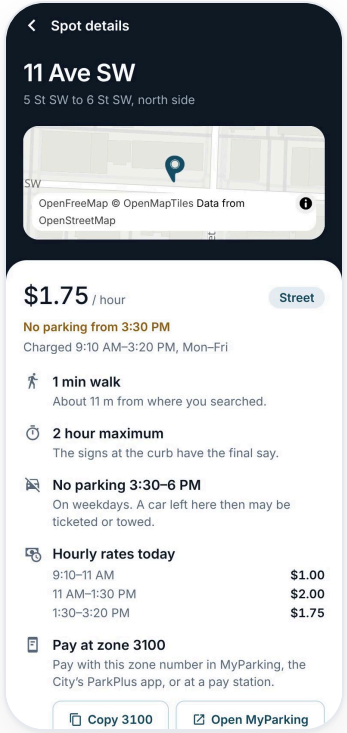
What was checked, and what wasn't

CHECK	HOW	RESULT
Logic	Jest unit tests: rates, rush hour, ranking, grouping and wording in the app; row merging, geometry and zone rates in the backend	152 app and 18 backend tests pass (run November 2, 2026)
Real data	1,672 City records within 1.5 km of 11 Ave SW and 5 St SW, served through the backend's own row code	Used for every screen and recording here
Map density	Count pins and overlaps on the opening web view	45 → 15 pins; 44 overlapping → 4 touching
Contrast	WCAG ratios for text on selected rows	Grey 4.23 → 4.74 : 1; amber warning 4.63 : 1
Keyboard	Tab through the web map screen	Pins skipped (the list has the same places); search is the first stop
Small phone	Web build at 360 × 640	Add time and End session moved from 676 and 771 px (off screen) to 263 and 358 px
Builds	Web build checked at 360, 390 and 1280 px wide; Android bundle exported	Both build without errors

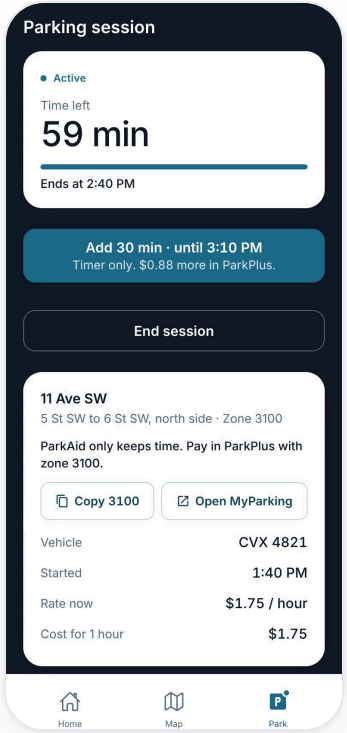
Not done yet

- No usability sessions with drivers.
- No testing on a physical phone in this round. Android was bundled, not run on a device.
- The database-backed suites (API contract, data quality, performance) need a database connection and weren't run here.
- Query speed wasn't measured. The README's "about 30 ms" is an older figure.

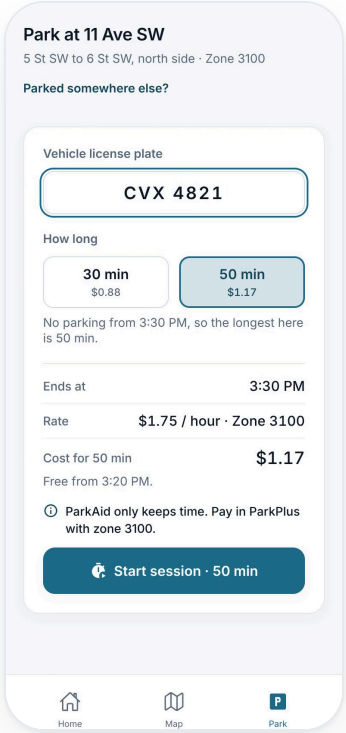
Seven recordings of the real build



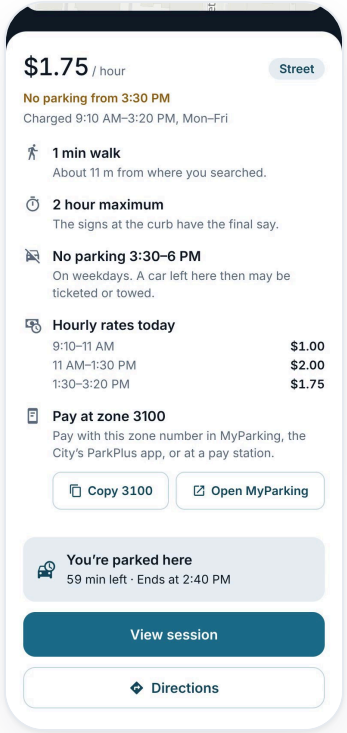
3 · Details. Rate now, rush hour, the day's rates, the zone to pay.



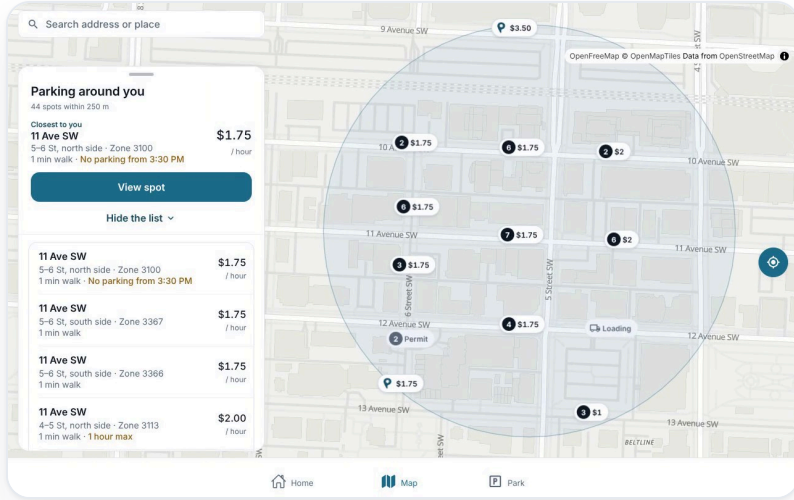
4 · Parking. Time left, add time, end, and where to pay.



5 · Rush hour. At 2:40 PM the longest stay is 50 min.



6 · Parked here. Details shows the time left on your own spot.



7 · Laptop. The map opens with its list beside it.

1 · Finding parking and **2 · The map** are on page 8.

Recorded with Playwright from the web build, at a fixed time, with the City's records. A blue circle marks each tap. MP4 and GIF:

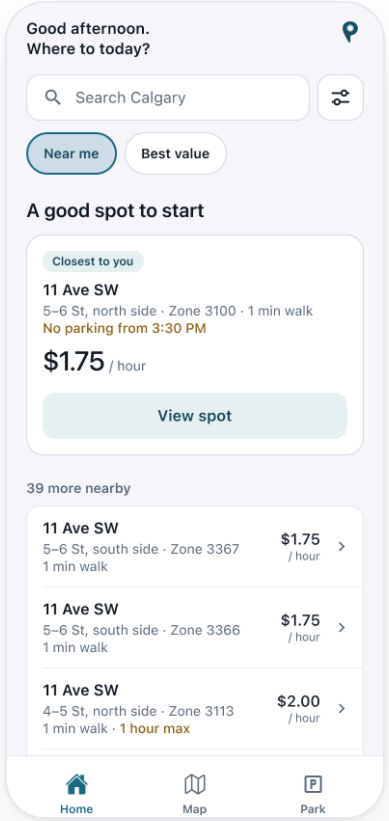
github.com/ginesbal/parkaid/tree/main/docs/walkthrough

The shipped interface, rebuilt as a design library

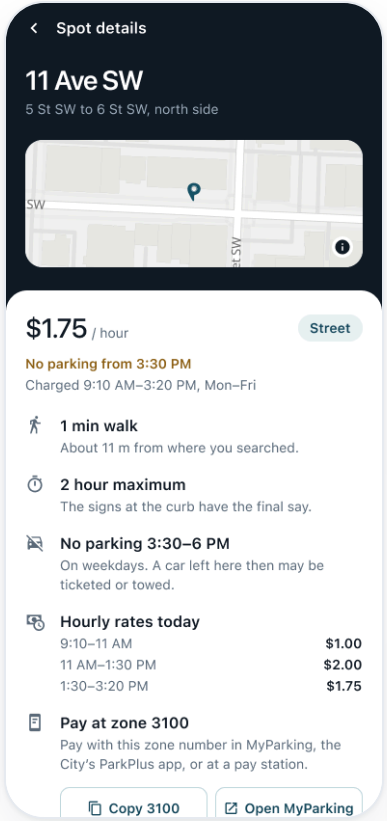
Made in November 2026 from the code, after the app was built, so the design can be inspected and reused. It isn't the original design source.

- 61 variables (colour, spacing, radius) and 14 text styles, matching the app's tokens.
- 26 icons imported from the icon sets the app uses, not redrawn.
- 11 components with variants and properties: buttons, chips, spot rows, map pins, tab bar and more.
- Screens so far: Welcome, Home, Map, Details, Park. The session screens and click-through links are still to add.

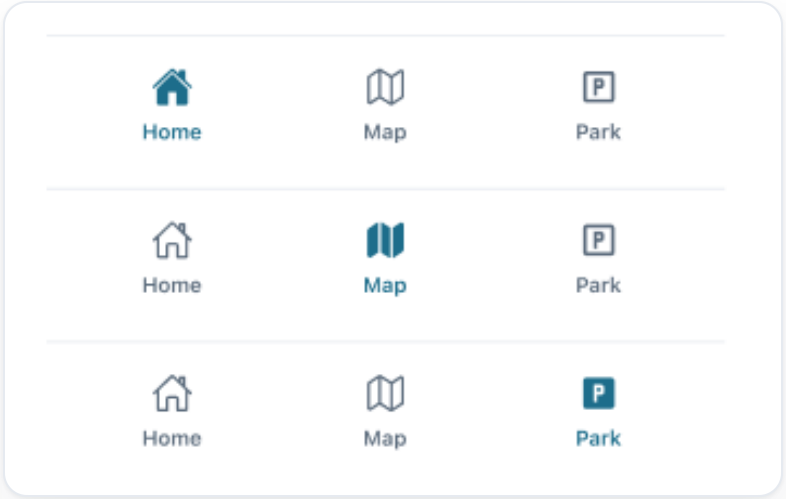
figma.com/design/dLkYCXg0Xzfm4NAGWAwcJF/ParkAid



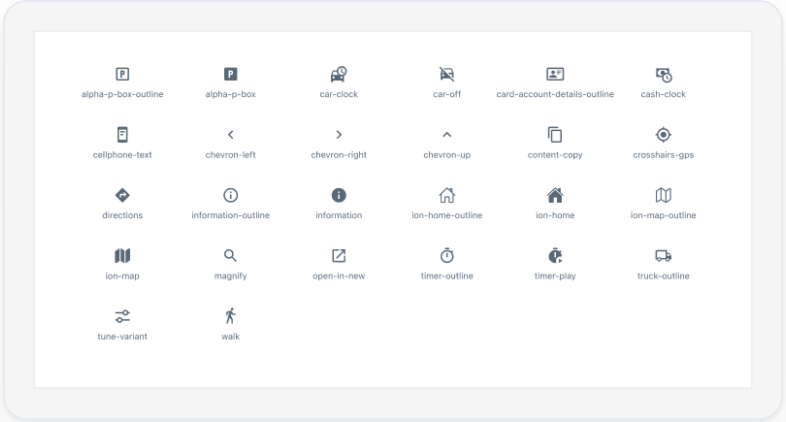
Home, built from component instances: search, chips, spot rows, tab bar.



Details, with the map preview from OpenStreetMap tiles.



Tab bar, one variant per active tab.



Icons, with colour bound to a variable.

What improved, what's still open

What improved

- Prices match the City's rate for the hour, not a guess from the zone code.
- Rush-hour windows warn, cap the stay and block Start, so a tow-away hour is never shown as free.
- Rows read like the curb: block, side, zone, walk.
- The map opens readable: 15 pins instead of 45 piled up.
- The backend deploys on its own and returns each block once.
- Fixed for small phones, laptops, keyboard use and contrast.

Honest gaps

- No usability testing with drivers yet.
- Not run on a physical phone in this round.
- The live database still needs the new import. The recordings used a local copy of the API with the same code.
- The Figma file is partial: session screens and click-through links are still to add.
- The City's data drives the app, but the signs at the curb have the final say. The app says so.

AI assistance

Claude Code wrote most of the code from March 2026 (101 of 128 commits), and helped make the walkthrough recordings, the Figma file and the first draft of this case study. An early branch came from OpenAI Codex (April 2026).

I directed the work: what to fix, in what order, and what the app should promise drivers.

SOURCE CODE
github.com/ginesbal/parkaid

DEMO RECORDINGS
github.com/ginesbal/parkaid/tree/main/docs/walkthrough

FIGMA FILE
figma.com/design/dLkYCXg0Xzfm4NAGWAwcJF/ParkAid